

```
select * from film;
```

```
update film  
set release_year = 2002  
where film_id = 129;
```

```
select * from actor;
```

```
select * from language;
```

```
select * from film_actor;
```

```
-- film --- language => M:1
```

```
-- Q: Get the name of actors along with the name of the films they were part of.
```

```
-- film  
-- film_actor  
-- actor
```

```
select f.title as film_name, CONCAT(a.first_name, ' ', a.last_name) as actor_name  
from film f  
join film_actor fa  
on f.film_id = fa.film_id  
join actor a  
on fa.actor_id = a.actor_id;
```

```
-- Get data of all the actors who performed in 'ACADEMY DINOSAUR' film  
-- Get the data ordered by actor first_name
```

```
select f.title as film_name, CONCAT(a.first_name, ' ', a.last_name) as actor_name  
from film f  
join film_actor fa  
on f.film_id = fa.film_id  
join actor a  
on fa.actor_id = a.actor_id  
where f.title = 'ACADEMY DINOSAUR'  
order by a.first_name;
```

```
-- Get the count of all the films  
select count(*) -- count(*) counts the number of rows.  
from film;
```

```
-- Aggregate Functions: count, min, max, avg ....
```

```
-- Get the avg rental rate.
```

```
select avg(rental_rate)
from film;
```

-- Get the avg rental rate with the film title.

```
select title, avg(rental_rate)
from film;
```

-- Error Code: 1140. In aggregated query without GROUP BY, expression #1 of SELECT list contains nonaggregated column 'sakila.film.title'; this is incompatible with sql_mode=only_full_group_by

```
create table batches(
    id int primary key,
    name varchar(50) not null
);
```

```
select * from batches;
```

```
select * from students;
```

```
create table students (
    id int primary key,
    name varchar(50) not null,
    marks int not null,
    batch_id int,

    constraint fk_batch_id
    foreign key (batch_id) references batches(id)
);
```

```
insert into batches values (1, 'AIML');
insert into batches values (2, 'SDE');
insert into batches values (3, 'Agentic');
insert into batches values (4, 'DS');
insert into batches values (5, 'Data Eng');
insert into batches values (6, 'AIML');
```

```
insert into students values (1, 'A', 80, 1);
insert into students values (2, 'B', 80, 2);
insert into students values (3, 'C', 80, 3);
insert into students values (4, 'D', 80, 3);
insert into students values (5, 'E', 80, 1);
insert into students values (6, 'F', 80, 1);
insert into students values (7, 'G', 80, 4);
insert into students values (8, 'H', 80, 1);
insert into students values (9, 'I', 80, 2);
insert into students values (10, 'J', 80, 4);
```

```
insert into students values (11, 'K', 80, 1);
insert into students values (12, 'L', 80, 4);
insert into students values (13, 'M', 80, 1);
insert into students values (14, 'N', 80, 2);
insert into students values (15, 'O', 80, 4);
insert into students values (16, 'O', 60, 5);
insert into students values (17, 'O', 78, 6);
```

```
-- Q: Get the avg marks of all the students
select avg(marks)
from students;
```

```
select name, avg(marks)
from students;
```

```
-- Get the avg marks per batch
-- Group By
```

```
select batch_id, avg(marks)
from students
group by batch_id;
```

```
-- Get the avg marks per batch along with their batch names
```

```
select s.batch_id, avg(s.marks)
from students s
join batches b
on s.batch_id = b.id
group by s.batch_id;
```

```
select b.name, b.id, avg(s.marks)
from students s
join batches b
on s.batch_id = b.id
group by b.name;
```

```
-- Error Code: 1055. Expression #2 of SELECT list is not in GROUP BY clause and contains
nonaggregated column 'sakila.b.id' which is not functionally dependent on columns in
GROUP BY clause; this is incompatible with sql_mode=only_full_group_by
```

```
select b.name, b.id, avg(s.marks)
from students s
join batches b
on s.batch_id = b.id
group by b.name, b.id;
```

```
-- (1, A1ML)
```

- (2, SDE)
- (3, Agentic)
- (4, DS)
- (5, DE)
- (6, AIML)

```
select max(avg(marks))  
from students;
```

- Aggregate functions can't be nested.
- Error Code: 1111. Invalid use of group function

- Find the count of films released every year.

```
select release_year, count(*) as film_released  
from film  
group by release_year;
```

- Find the rental_duration which is rental_duration of more than 200 films.

```
select rental_duration, count(*) as total_films  
from film  
group by rental_duration  
having total_films > 200;
```

- having is used to filter groups.